COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	02/10/2026	**DATE SUBMITTED:**	02/10/2026
NAME:	Quiming, Kadaphy James A. Larracas, Jerry Matthew L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 10.1 GRAPHS

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

Add a vertex & Add an edge

```
#include <iostream>
// stores adjacency list items
  struct adjNode {
    int val, cost;
    adjNode* next;
    };
// structure to store edges
  struct graphEdge {
    int start_ver, end_ver, weight;
    };

  class DiaGraph{
// insert new nodes into adjacency list from given graph
adjNode* getAdjListNode(int value, int weight, adjNode* head) {
  adjNode* newNode = new adjNode;
  newNode->val = value;
  newNode->cost = weight;
  newNode->next = head; // point new node to current head

  return newNode;
  }

  int N; // number of nodes in the graph
  public:
  adjNode **head; //adjacency list as array of pointers

// Constructor
DiaGraph(graphEdge edges[], int n, int N) {
// allocate new node
head = new adjNode*[N]();
this->N = N;
// initialize head pointer for all vertices
for (int i = 0; i < N; ++i)
  head[i] = nullptr;
// construct directed graph by adding edges to it
  for (unsigned i = 0; i < n; i++) {
    int start_ver = edges[i].start_ver;
    int end_ver = edges[i].end_ver;
    int weight = edges[i].weight;
```

Display the graph vertices

```
// graph implementation
int main() {
// graph edges array.
graphEdge edges[] = {
// (x, y, w) -> edge from x to y with weight w
{0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
};

int N = 6; // Number of vertices in the graph
// calculate number of edges
int n = sizeof(edges)/sizeof(edges[0]);
// construct graph
DiaGraph diagraph(edges, n, N);
// print adjacency list representation of graph
std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;
  for (int i = 0; i < N; i++) {
// display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
  }
return 0;
}
```

Analysis: To add a vertex we need to create a class that indicates the nodes with values, weight, and head and adding edges creating a graph.

A.2. Adjacency Matrix

Code

```cpp
#include <iostream>
#define V 6

void initAdjMatrix(int adjMatrix[V][V]){
  for(int i = 0; i < V; i++){
    for(int j = 0; j < V; j++){
      adjMatrix[i][j] = 0;
    }
  }
}
void addEdge(int adjMatrix[V][V], int u, int v){
  adjMatrix[u][v] = 1;
  adjMatrix[v][u] = 1;
}
void printAdjMatrix(int adjMatrix[V][V]){
  for(int i = 0; i < V; i++){
    for(int j = 0; j < V; j++){
      std::cout << adjMatrix[i][j] << " ";
    }
    std::cout << std::endl;
  }
}
```

```cpp
int main()
{
    int adjMatrix[V][V];

    initAdjMatrix(adjMatrix);

    addEdge(adjMatrix, 0, 3);
    addEdge(adjMatrix, 0, 4);
    addEdge(adjMatrix, 1, 4);
    addEdge(adjMatrix, 1, 5);
    addEdge(adjMatrix, 1, 2);
    addEdge(adjMatrix, 2, 5);
    addEdge(adjMatrix, 2, 3);
    addEdge(adjMatrix, 3, 5);
    addEdge(adjMatrix, 4, 5);

    std::cout << "Adjacency Matrix:" << std::endl;
    printAdjMatrix(adjMatrix);

    return 0;
}
```

Output

```
Adjacency Matrix:
0 0 0 1 1 0
0 0 1 0 1 1
0 1 0 1 0 1
1 0 1 0 0 1
1 1 0 0 0 1
0 1 1 1 1 0
```

Analysis: The code creates and displays an adjacency connecting vertices through edges represented by 1 and 0.

A.3. Adjacency List

Code

```cpp
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
  while (ptr != nullptr) {
    std::cout << "(" << i << ", " << ptr->val
    << ", " << ptr->cost << ") ";
    ptr = ptr->next;
    }
    std::cout << std::endl;
    }
// graph implementation
int main() {
// graph edges array.
graphEdge edges[] = {
// (x, y, w) -> edge from x to y with weight w
{0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
};

int N = 6; // Number of vertices in the graph
// calculate number of edges
int n = sizeof(edges)/sizeof(edges[0]);
// construct graph
DiaGraph diagraph(edges, n, N);
// print adjacency list representation of graph
std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;
  for (int i = 0; i < N; i++) {
// display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
  }
return 0;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Analysis: This creates and displays an adjacency list where each vertex is connected to another vertex with a specified edge weight.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

1	Include the required header files, as follows:

```
1  // ILO B
2  #include <string>
3  #include <vector>
4  #include <iostream>
5  #include <set>
6  #include <map>
7  #include <stack>
8
9  template <typename T>
10 class Graph;
```

Header code

2	Write the following struct in order to implement an edge in our graph:

```cpp
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

Edge struct stores src/dest/weight with operator overloads to compare edges by weight

3	Next, overload the << operator for the graph so that it can be printed out using the following function:

```cpp
// STEP 3
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

Stream operator prints all vertices and their outgoing edges

4	Implement the graph data structure that uses an edge list representation as follows:

```cpp
template <typename T>
class Graph
{
  public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

  private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

Graph class manages V vertices and edge_list, with add_edge() validation and outgoing_edges() retrival

5	Now, we need a function to perform DFS for our graph. Implement it as follows:

```cpp
// STEP 5 : DFS
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

DFS uses a stack to traverse from vertex 1, marking visited notes and recording visit order

6	We shall test our implementation of the DFS on the graph shown here

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Creates 9-vertex reference graph.

7	Finally, add the following test and driver code, which runs our DFS implementation and prints the output:

```cpp
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
```

Tests the code

8	**Compile and run the preceding code. Include your output as a table in section 6.**

```
PS C:\Users\TIPQC\Documents\Quiming-Temp> cd "c:\Users\TIPQC\Documents\Quiming-Temp\" ; if ($?) { g++ ILOB.cpp -o ILOB } ; if ($?) { .\ILOB }
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

1	Include the required header files and declare the graph as follows

```cpp
// STEP 1
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;
```

Header code

2	Write the following struct, which represents an edge in our graph

```cpp
// STEP 2
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

Edge struct stores src/dest/weight with operator overloads to compare edges by weight

3	Next, overload the << operator for the Graph data type in order to display the contents of the graph

```cpp
// STEP 3
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
    return os;
}
```

4	Write a class to define our graph data structure, as shown here:

```cpp
// STEP 4
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

Stream operator prints all vertices and their outgoing edges

5	For this exercise, we shall test our implementation of BFS on the following graph:

```cpp
// STEP 5
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

  return G;

}
```

Creates 9 vertices.

6	Implement the breadth-first search like so:

```cpp
// STEP 6
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;

    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

DFS uses queue to traverse from vertex 1 then marking visited nodes

7	Add the following test and driver code that creates the reference graph, runs BFS starting from vertex 1, and outputs the results:

```cpp
// STEP 7
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    std::cout << "\nCode is Working!";
    return 0;
}
```

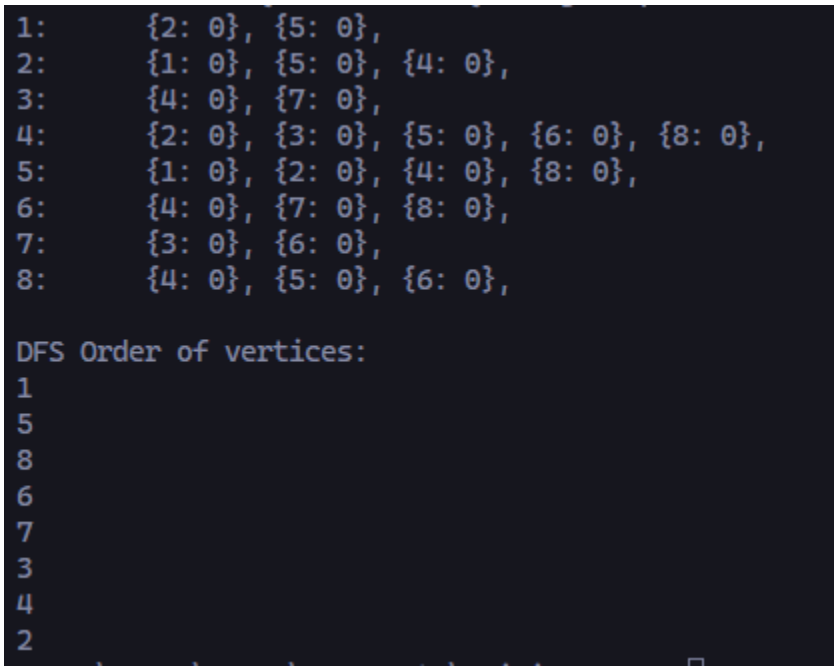
Test the test_bfs

```
PS C:\Users\TIPQC\Documents\Quiming-Temp> cd "c:
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

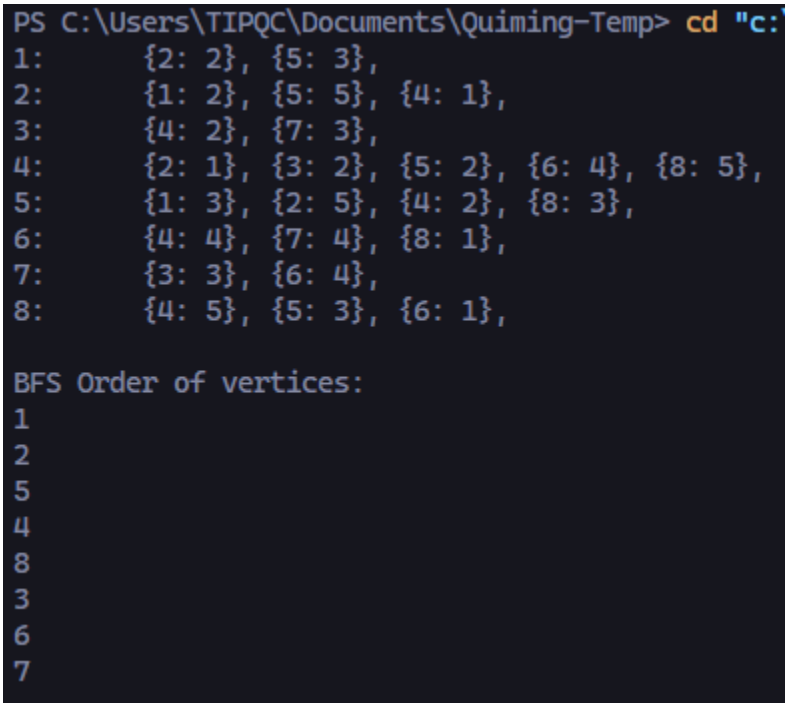
BFS Order of vertices:
1
2
5
4
8
3
6
7
```

OUTPUT

ILO B1

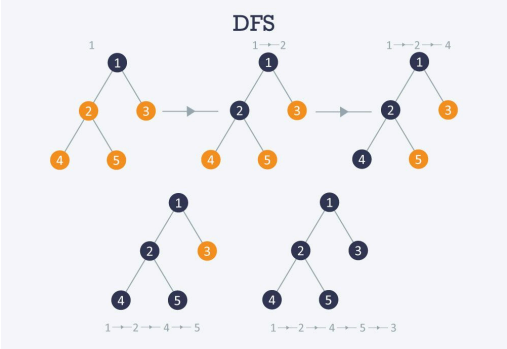
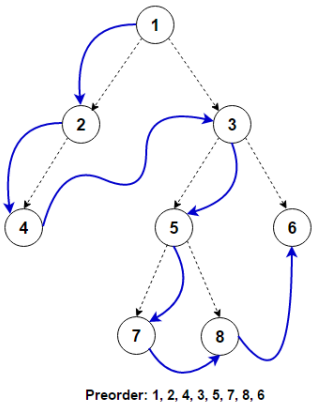


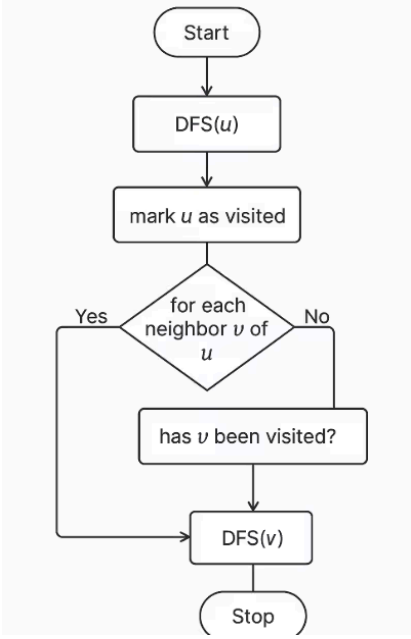
ILO B2



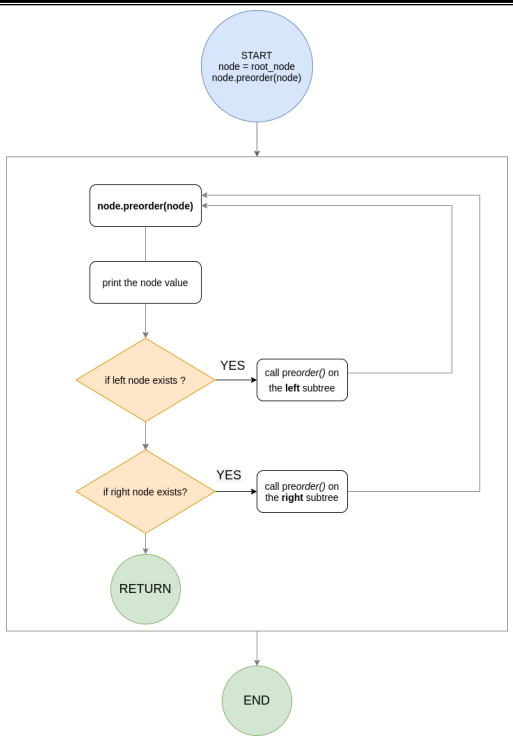
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.	
1	A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
The best algorithm to achieve that person's goal is the Depth-First Search or the DFS. He starts at one location then visits every **unvisited** neighboring vertex as deep as he can go. Whenever theres a dead end, he then backtracks to the previous location(vertex) and checks for unexplored/unvisited neighboring location, repeating this strategy until he finished visiting all the location on the map.	
2	Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
The equivalent of DFS in traversal strategies for trees is the **pre-order** strategy.	

Graph's DFS	Tree's Pre-Order
 Depth First Search Algorithm | Board Infinity	 Preorder Traversal in 2 Ways | Maxnilz



[Depth First Search: What It Is & How's It Work in 2026]



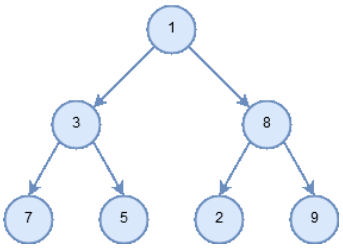
[https://levelup.gitconnected.com/how-to-traverse-a-tree-using-javascript-c9a79826e819]

```cpp
// STEP 5 : DFS
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
void BINARY_SEARCH_TREE<T>::PRE_ORDER(){
    std::cout << KEY << "    ";
    if(LEFT != nullptr){
        LEFT -> PRE_ORDER();
    }

    if(RIGHT != nullptr){
        RIGHT -> PRE_ORDER();
    }
}
```

They have similar algorithms too, they both travel to the furthest node then backtracks.

9.15. General Depth First Search — Problem Solving with Algorithms and Data Structures using C++	 8.6. Tree Traversals — Problem Solving with Algorithms and Data Structures using C++

3	In the performed code, what data structure is used to implement the Breadth First Search?
The data structure that was used to implement BFS was a queue which includes push and pop of finding or searching for vertices.	

4	How many times can a node be visited in the BFS?
A node can be visited only once in the BFS.	

CONCLUSION	
Larracas	
Quiming	In this activity, I learned how to implement graphs using adjacency lists and edge lists, and that DFS(Stack) and BFS(Queue) each visits every vortex/node once.

Reference for Adjacency Matrix implementation:
https://www.geeksforgeeks.org/cpp/cpp-program-to-implement-adjacency-matrix/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n/a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n/a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n/a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.